

AI ENGINEERING • RAG & LLM EVALUATION

"It feels better" is **not** a metric.

How to actually prove your RAG or LLM app works: golden sets, metrics, LLM judges, CI gates and production monitoring, end to end.

THE DEMO

10 questions ✓

Hand-picked. All look perfect.

PRODUCTION

10,000 questions ?

Real users. Nobody is checking.

TOOLS COVERED

Ra

RAGAS

TI

TruLens

De

DeepEval

Pf

Promptfoo

Ls

LangSmith

Lf

Langfuse

Ph

Phoenix

MI

MLflow

THE PROBLEM

Why evaluating LLM apps is hard

No single right answer

Two answers can both be correct but worded completely differently.

Non-deterministic

The same question can give a different answer on the next run.

Many places to fail

Parsing, chunking, retrieval, ranking, prompting, generation.

Fails silently

Wrong answers still sound fluent and confident. Nothing throws an error.

THE "VIBE CHECK" TRAP

Most teams test 5–10 questions by hand, tweak something, test again, and ship when it "feels better". That catches obvious breaks, but it can't tell you whether a change helped 3% or quietly broke 30 other questions.

THE GOAL

Good evaluation turns "I think it works" into a number you can track, compare and gate releases on.

THE BIG PICTURE

What to evaluate: three layers

1

Retrieval

Did we find the right chunks? Recall, precision, ranking.

2

Generation

Is the answer grounded, relevant, correct and complete?

3

System

Latency, cost per query, safety, refusals, user satisfaction.

THE RAG TRIAD (POPULARISED BY TRULENS)

Context relevance

Question ↔ retrieved chunks

Groundedness

Chunks ↔ answer

Answer relevance

Answer ↔ question

If all three pass, the answer is very likely trustworthy. If one fails, you know exactly where to look.

STEP 1 • THE GOLDEN SET

Build a golden set first

A fixed list of real questions with known answers. Everything else is measured against it.

```
{
  "id": "q_042",
  "question": "Can I get a refund after 30 days?",
  "expected_answer": "Only for annual plans, pro-rated.",
  "source": { "doc": "refund_policy.pdf", "page": 3 },
  "slice": "policy · exception · multi-hop"
}
```

WHERE QUESTIONS COME FROM

- ✓ Real user queries from logs (best)
- ✓ Subject-matter experts
- ✓ Synthetic questions, reviewed by a human

WHAT TO INCLUDE

- ✓ Easy, hard and multi-hop questions
- ✓ **Unanswerable** ones to test refusal
- ✓ Time-sensitive and exact-ID questions

Start with 30–50 before tuning anything. Grow it to 200+ by adding every real production failure as a new case.

STEP 2 • RETRIEVAL METRICS

Did we find the right chunks?

Recall@k

Did the relevant chunk appear in the top k at all? The most important retrieval metric.

Precision@k

Of the k chunks retrieved, how many were actually useful? Low precision = noisy context.

MRR

How high did the first correct chunk rank? 1st place = 1.0, 2nd = 0.5, 3rd = 0.33.

nDCG

Rewards putting the most relevant chunks first, when several are relevant.

SMALL EXAMPLE

#1 x

#2 ✓

#3 x

#4 ✓

#5 x

2 relevant chunks exist, both found in the top 5: **Recall@5 = 1.0**. 2 of 5 useful: **Precision@5 = 0.4**. First hit at #2: **MRR = 0.5**.

STEP 3 • GENERATION METRICS

Is the answer actually good?

Faithfulness

Is every claim supported by the retrieved context?

Answer relevance

Does it answer what was asked, not something nearby?

Correctness

Does it match the expected answer?

Completeness

Did it cover every part of the question?

Citation accuracy

Does each cited source really support its claim?

Refusal correctness

Says "I don't know" when it should, and only then.

SMALL EXAMPLE: FLUENT BUT UNFAITHFUL

Context: "Refunds within 30 days. Annual plans: pro-rated after that."

Answer: "Refunds are available within 30 days, or 60 days for premium users."

Relevant and fluent, but the highlighted part appears nowhere in the context. Faithfulness catches it.

STEP 4 • HOW TO SCORE

Four ways to score an answer

METHOD	GOOD FOR	WATCH OUT
Exact match / F1	IDs, numbers, short facts	Fails on correct rewording
Embedding similarity	Cheap semantic closeness	Similar ≠ correct
LLM-as-judge	Faithfulness, relevance, tone, at scale	Has biases, needs calibrating
Human review	The gold standard, edge cases	Slow and expensive

IN PRACTICE

Real teams mix them: **exact match** for facts, an **LLM judge** for the bulk, and **humans** on a sample to keep the judge honest.

STEP 5 • LLM-AS-JUDGE

Using an LLM as the judge, safely

You are grading an answer for FAITHFULNESS.

Context: {context}

Answer: {answer}

List each claim in the answer.

For each claim: SUPPORTED or NOT_SUPPORTED by the context.

Return JSON: {"claims": [...], "score": supported / total}

DO THIS

- ✓ Use a clear rubric, not "rate 1–10"
- ✓ Binary or 3-point scales
- ✓ Ask for reasons before the score
- ✓ Judge with a different model family

KNOWN JUDGE BIASES

- ✗ **Position bias:** favours the first option
- ✗ **Verbosity bias:** favours longer answers
- ✗ **Self-preference:** favours its own outputs

THE RULE

Calibrate before trusting it: label ~100 answers by hand and check how often the judge agrees. Those biases were documented in the 2023 "Judging LLM-as-a-Judge" (MT-Bench) paper.

TRAPS

Five ways evals lie to you

1. Pretraining leak

The model already knew the answer, so retrieval looks fine. Fix: a closed-book baseline with no context.

2. Overfitting the eval set

Tuning against the same questions you report on. Fix: a held-out set you never tune on.

3. Small-sample noise

With 50 questions, one flip = 2%. Fix: compare question by question, not just the average.

4. One average hides slices

Great overall, terrible on exact-ID or multi-hop questions. Fix: score per slice.

5. A stale golden set

Users ask new things; the set doesn't. Fix: refresh it with recent production queries.

STEP 6 • OFFLINE EVALS IN CI

Gate every change like a test

Change chunking / prompt



Run golden set



Compare to baseline



Pass → ship

Fail → block

```
# DeepEval: an LLM eval written like a pytest unit test
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import FaithfulnessMetric

def test_refund_policy():
    case = LLMTestCase(
        input="Can I get a refund after 30 days?",
        actual_output=rag_app("Can I get a refund after 30 days?"),
        retrieval_context=retrieve("refund after 30 days"),
    )
    assert_test(case, [FaithfulnessMetric(threshold=0.8)])
```

Version everything

Chunker, chunk size, embedding model, prompt and the golden set itself.

Gate per slice

Block a release if any important slice drops, not just the average.

STEP 7 • PRODUCTION

Keep evaluating after launch

Offline evals catch regressions before release. Production monitoring catches the silent drift that comes later.

TRACE EVERY REQUEST

- ✓ Query, retrieved chunks, prompt, answer
- ✓ Latency and cost per step
- ✓ Tools: Langfuse, LangSmith, Phoenix

SCORE A SAMPLE DAILY

- ✓ LLM-judge groundedness on ~1–5%
- ✓ Human review of the worst cases
- ✓ New failures → golden set

EARLY WARNING SIGNALS

top-1 similarity dropping

more "no chunk found"

rising "I don't know" rate

thumbs-down

users rephrasing

p95 latency

No single signal proves a problem, but several moving together usually means retrieval is degrading.

TOOLS

The evaluation toolkit



RAGAS
OPEN SOURCE

RAG metrics: faithfulness, context precision and recall



TruLens
OPEN SOURCE

Feedback functions and the RAG triad



DeepEval
OPEN SOURCE

LLM evals as pytest-style unit tests in CI



Promptfoo
OPEN SOURCE

CLI for prompt comparisons, regression tests and red-teaming



LangSmith
HOSTED

Tracing, datasets and evaluators, from the LangChain team



Langfuse
OPEN SOURCE

LLM tracing, observability and evals, self-hostable



Phoenix
OPEN SOURCE

Arize's tracing and evaluation for LLM apps



MLflow
OPEN SOURCE

Experiment tracking, so every index change becomes a run

[START HERE](#)

A 4-week plan for any team

W1

Build the golden set

30–50 real questions with expected answers and source passages, including unanswerable ones.

W2

Measure retrieval

Recall@k and MRR per slice. Fix chunking and search before touching prompts.

W3

Add a calibrated judge

Faithfulness and relevance scoring, checked against ~100 human labels.

W4

Gate and monitor

Run evals in CI on every change, then trace and sample production daily.

REMEMBER

You don't need a perfect setup on day one. A small, honest golden set beats a perfect plan you never build.

RECAP

LLM evaluation in one slide

- 1 Vibe checks can't catch silent regressions
- 2 Evaluate retrieval, generation and system separately
- 3 Start with a 30–50 question golden set
- 4 Recall@k for retrieval, faithfulness for answers
- 5 Calibrate your LLM judge against humans
- 6 Gate every change in CI, per slice
- 7 Keep monitoring after launch

Follow Viresh Navtake

I break down one AI or system design topic every week.
Follow so you don't miss the next one.

Save this for your next RAG build.